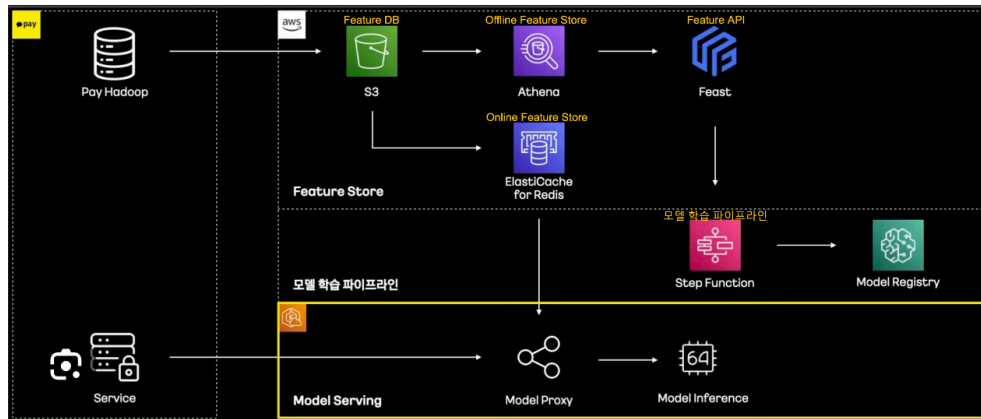




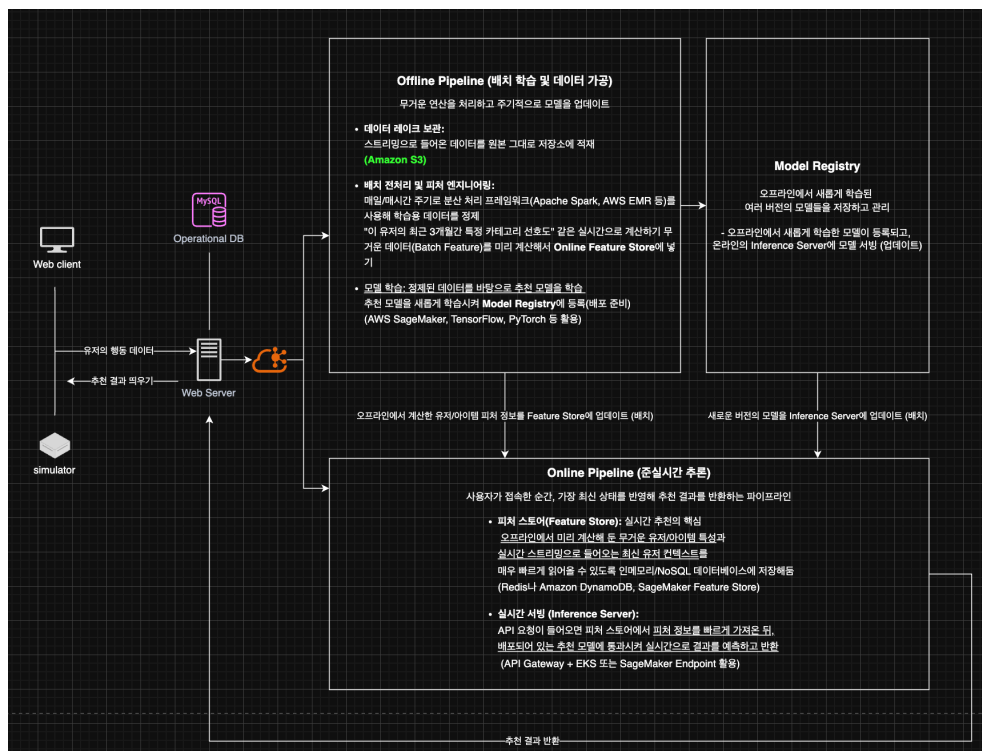
파이프라인 (1학기기말 ver)

☼ 상태	완료
☰ 다중 선택	

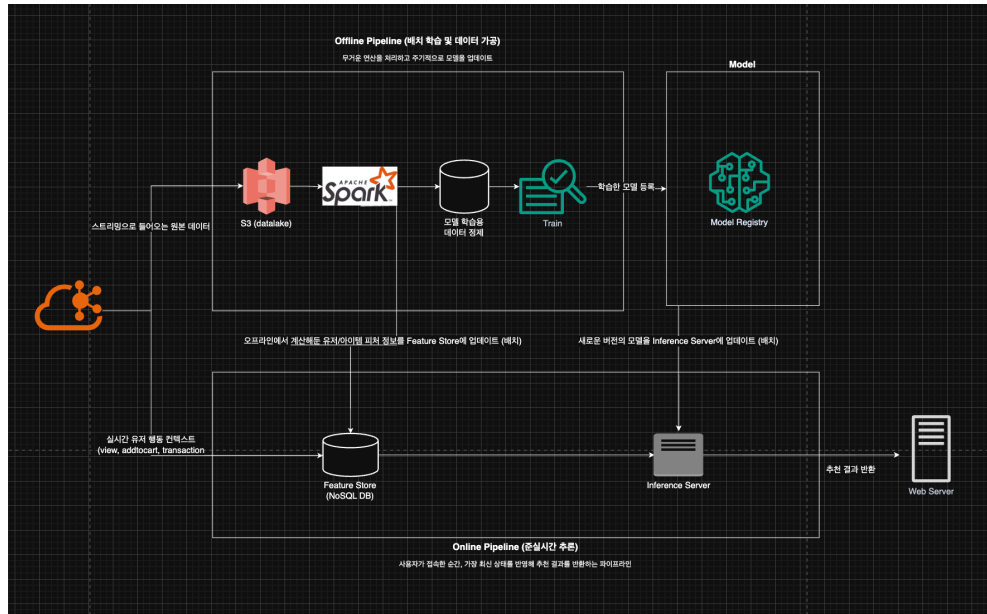
ex) 카카오페이 사례



- 초안



- 선택안 1) 쿠팡/넷플릭스처럼 초당 수천~수만 건의 데이터가 쏟아지는 거대한 환경을 위한 형태



- 딥러닝 기반 추천 (BERT4Rec 등) 을 전제로 한 구조
 - 딥러닝은 실시간으로 무거운 연산이 필요하니까 Flink, Feature Store가 필요

준실시간 추천 파이프라인

준실시간(Near Real-time) 추천 파이프라인은 크게 아래의 3가지로 나누어 설계된다

- 오프라인 파이프라인(학습용)
- 온라인 파이프라인(서빙용)
- 이를 이어주는 **MLOps 및 Feature Store**

일반적으로 구성하는 표준적인 파이프라인 아키텍처를 단계별로 정리

1. Raw Data 생성

2. 데이터 수집 및 스트리밍 (Data Ingestion)

사용자의 실시간 행동 데이터(클릭, 조회, 구매 등)를 수집하는 관문

- 스트리밍 처리: 애플리케이션에서 발생하는 로그를 즉각적으로 받아주는 **메세지 큐(Message Queue)**를 사용
주로 **Apache Kafka**나 **Amazon Kinesis**, **AWS MSK** 같은 서비스를 활용하여 대량의 실시간 데이터를 병목 없이 수용
 - 메세지 큐 : 유저의 행동 데이터가 들어오면 메세지 큐에서 데이터를 양쪽으로 동시에 뿌려줌.
 1. **영구 저장 → S3**: 동시에 Kafka Connect나 Kinesis Firehose 같은 도구가 원본 데이터를 그대로 묶어서 S3(Data Lake / Offline Feature Store)에 영구적으로 쌓아둠.
 2. **실시간 처리 → Feature Store**: 스트리밍 처리기(Flink 등)가 데이터를 받아서 즉시 가공한 뒤, 실시간 서빙을 위해 Redis(Online Feature Store)에 꽂아 넣음

3. 오프라인 파이프라인 (배치 학습 및 데이터 가공)

이 파이프라인은 무거운 연산을 처리하고 주기적으로 모델을 업데이트하는 역할

1. **데이터 레이크 보관**: 스트리밍으로 들어온 데이터를 원본 그대로 저장소(Amazon S3)에 적재
 - 모델 학습(Offline Feature Store) : 모델을 배치로 학습시키려면 과거부터 지금까지의 방대한 피쳐 데이터 히스토리가 필요
 - 영구 저장

- S3에 데이터가 쌓이지 않으면 모델을 재학습시킬 때 쓸 과거 데이터가 없음

2. 배치 처리 및 피처 엔지니어링:

주기적으로 분산 처리 프레임워크(Apache Spark, AWS EMR 등)를 사용해 학습용 데이터를 정제

2-1. 새벽에 **Spark 같은 배치 잡(Batch Job)**이 S3 데이터를 짝 끊어서 무거운 통계 연산을 돌리고

→ 그 결과값만 **Online Feature Store**에 업데이트한다

2-2. **모델 학습**: 정제된 데이터를 바탕으로 추천 모델을 학습

- (AWS SageMaker, TensorFlow, PyTorch 등 활용)

→ 추천 모델을 새롭게 학습시켜 **Model Registry**에 등록(배포 준비)

4. 온라인 파이프라인 (준실시간 추론)

사용자가 접속한 바로 그 순간, 가장 최신 상태를 반영해 추천 결과를 반환하는 파이프라인

1. **스트리밍 처리기(Flink 등)**가 데이터를 받아서 즉시 가공한 뒤, 실시간 서빙을 위해 Redis(Online Feature Store)에 꽂아 넣음

2. **Online Feature Store** : 실시간 추천의 핵심(실시간 서빙용)

오프라인에서 미리 계산해 둔 무거운 유저/아이템 특성과 실시간 스트리밍으로 들어오는 최신 유저 컨텍스트(방금 클릭한 상품)를 매우 빠르게 읽어올 수 있도록

Redis나 Amazon DynamoDB, SageMaker Feature Store 같은 **인메모리/NoSQL 데이터베이스**에 저장해둠

- 만약 인메모리 DB가 날아갔을 때, S3에 저장된 원본(Raw Data)을 다시 읽어와서 Feature Store를 원상 복구(Replay)

- **실시간 서빙 (Inference Endpoint)**:

API 요청이 들어오면 피쳐 스토어에서 유저 정보를 초고속으로 가져온 뒤, 배포되어 있는 추천 모델에 통과시켜 실시간으로 결과를 예측하고 반환합니다. (API Gateway + EKS 또는 SageMaker Endpoint 활용)

실시간 추천 서빙

1. 앱에서 Web Server로 "지금 이 유저에게 보여줄 추천 리스트 줘!"라고 요청하면, Web Server는 **Inference Server(추천 추론 API)**를 찌릅니다.
 - Inference Server는 즉시 **Online Feature Store**에 접근해 해당 유저의 '무거운 피쳐(배치 통계)'와 '가벼운 피쳐(실시간 컨텍스트)'를 모두 가져옵니다
 - 이 피쳐 데이터를 방금 배포된 최신 모델에 넣어 실시간으로 점수를 매기고, 가장 적합한 결과를 Web Server를 거쳐 유저에게 반환

5. MLOps 및 모델 레지스트리 (지속적 모델 변경)

계속해서 더 나은 모델을 실험하고, 서비스 중단 없이 교체하기 위한 관리 영역

- **모델 레지스트리**:

파이프라인에서 쏟아져 나오는 수많은 모델 파일들을 안전하게 보관하고, 버전별로 족보를 관리하며, 배포 상태를 추적하는 **중앙 저장소** 역할

- 오프라인에서 새롭게 학습된 여러 버전의 모델들을 저장하고 관리
- (MLflow, SageMaker Model Registry 등)

- **배포 전략 (A/B 테스트 및 새도우 배포)**:

기존 모델(A)과 새로 개발한 타겟 모델(B)의 성능을 비교하고,

- 트래픽을 분산시키거나,
- 유저에게는 기존 결과를 보여주되 백그라운드에서는 새 모델의 예측값을 기록(Shadowing)하여 실험 환경 성능을 검증한 뒤

→ 안전하게 교체
